

Sistemas Operativos Fisop 2025

El Proceso

De Programa a Proceso

Conceptos clave:

- Proceso de compilación
- Object files y ejecutables
- Linker y Loader
- Formato ELF

El Programa

1. Programador edita código fuente
2. El compilador compila el source code en una secuencia de instrucciones de máquina.
3. El compilador guarda en disco esa secuencia, junto con datos y metadata del programa: programa ejecutable.

El Programa: Edición

```
#include<stdio.h>
```

```
#define FORMAT_STRING "%s"
```

```
#define MESSAGE "Hello , World\n"
```

```
int main(){
```

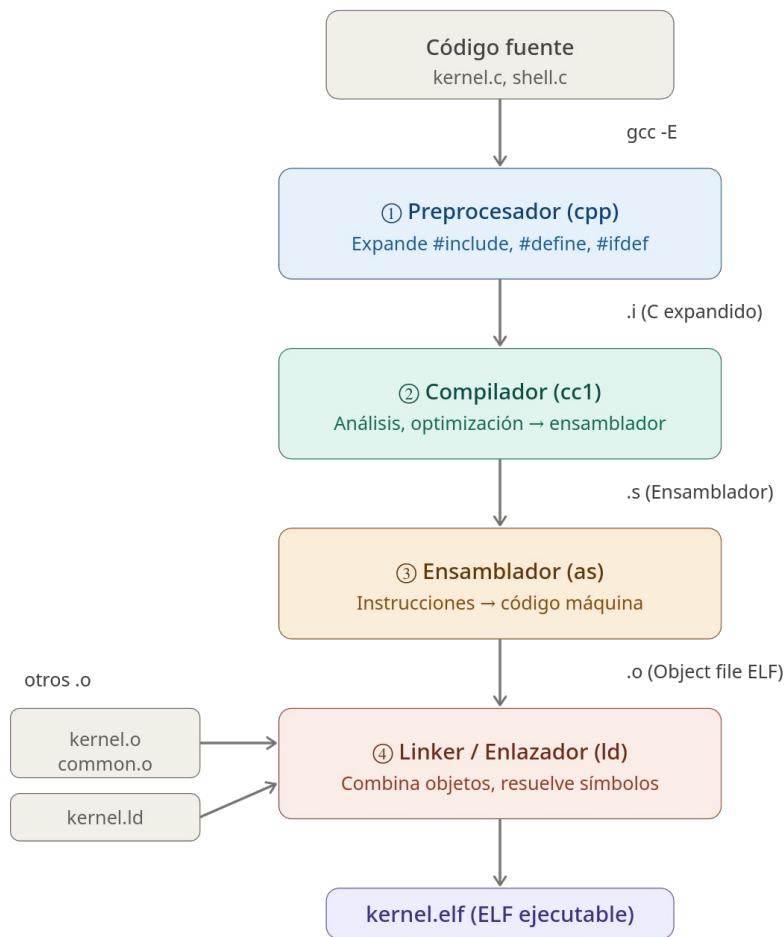
```
    printf(FORMAT_STRING,MESSAGE);
```

```
    return 0;
```

```
}
```

#	i	n	c	l	u	d	e	<sp>	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	<sp>	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	<sp>	<sp>	<sp>	<sp>	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	<sp>	w	o	r	l	d	\	n	"	)	;	\n	}
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	125

El Programa: Compilación



El proceso de transformar código fuente en un binario ejecutable no es un paso único. Es una cadena de cuatro herramientas distintas:

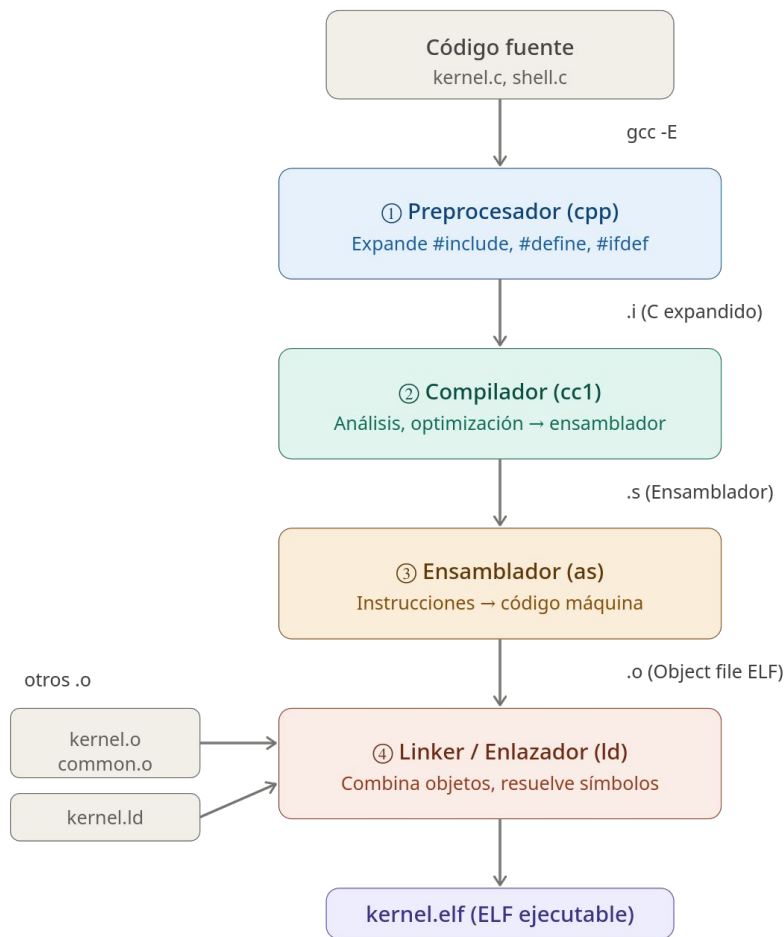
Fase 1 — El Preprocesador (cpp)

Esta es la fase más "textual" de todas. El preprocesador no sabe nada de C como lenguaje: trabaja puramente sobre el texto del archivo fuente. Su trabajo es producir una unidad de traducción limpia antes de que el compilador real entre en juego.

Por ejemplo:

`#include`: Reemplaza la directiva por el contenido completo del archivo referenciado. Cuando en `kernel.c` escribís `#include "kernel.h"`, el preprocesador literalmente copia el contenido de `kernel.h` en ese punto. Esto significa que al compilador le llega un único archivo gigante con todo adentro.

El Programa: Compilación



FASE 2- El Compilador propiamente dicho (cc1)

Aquí ocurre la mayor parte del "trabajo inteligente". El compilador toma el `.i` y produce código ensamblador (`.s`). Esta fase tiene varias subfases internas:

Análisis léxico: El compilador descompone el texto en tokens (palabras reservadas, identificadores, operadores, literales). `uint32_t`, `=`, `0x80200000`, `;` son cada uno un token.

Análisis sintáctico (parsing): Construye un árbol de sintaxis abstracta (AST). Verifica que la gramática del lenguaje se cumpla. Aquí se detectan errores como `if (a = b` sin cerrar el paréntesis.

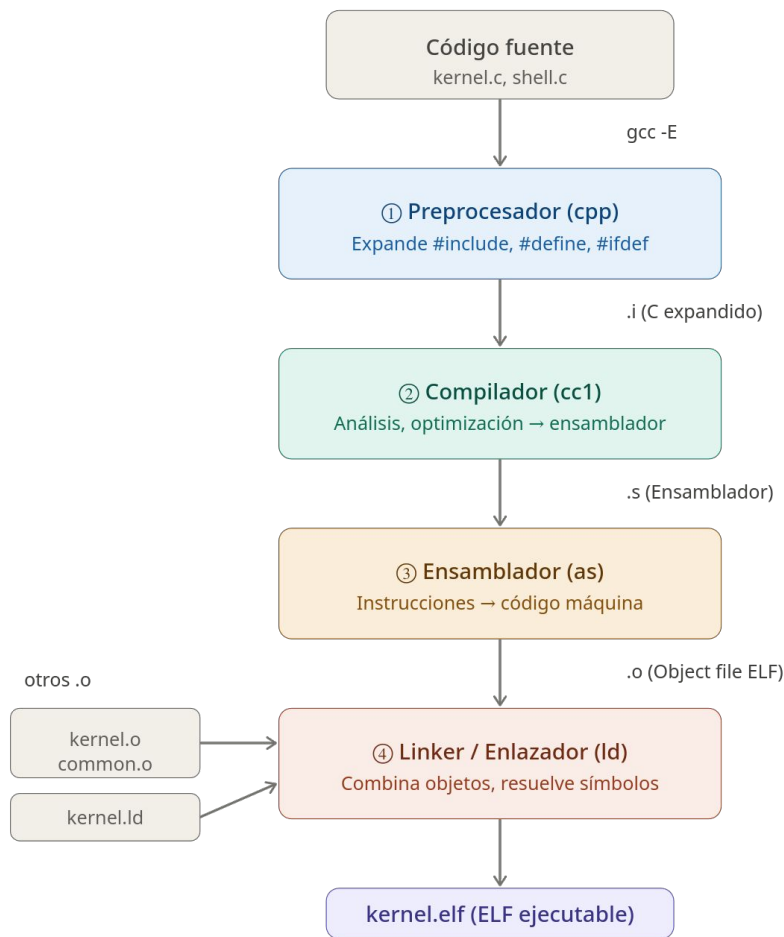
Análisis semántico: Verifica tipos, scoping, que las funciones declaradas existan, etc. Si llamas a una función con argumentos del tipo incorrecto, el error aparece aquí.

Generación de código intermedio: Muchos compiladores (como GCC o Clang/LLVM) no pasan directamente a ensamblador, sino que generan una representación intermedia (IR) agnóstica de la arquitectura. LLVM usa su propio IR; GCC usa GIMPLE/RTL.

Optimización: La fase más compleja. El compilador puede reordenar instrucciones, eliminar código muerto, hacer inlining de funciones, propagar constantes, etc. Los niveles `-O0`, `-O1`, `-O2`, `-O3` controlan cuánto se optimiza (a mayor nivel, más tiempo de compilación pero binario más eficiente).

Selección de instrucciones y generación de ensamblador: Finalmente se genera el `.s` para la arquitectura objetivo. En el contexto de los libros del proyecto (RISC-V 32-bit), el compilador genera instrucciones como `addi`, `lw`, `sw`, `jal`, etc.

El Programa: Compilación



Fase 3 — El Ensamblador (as)

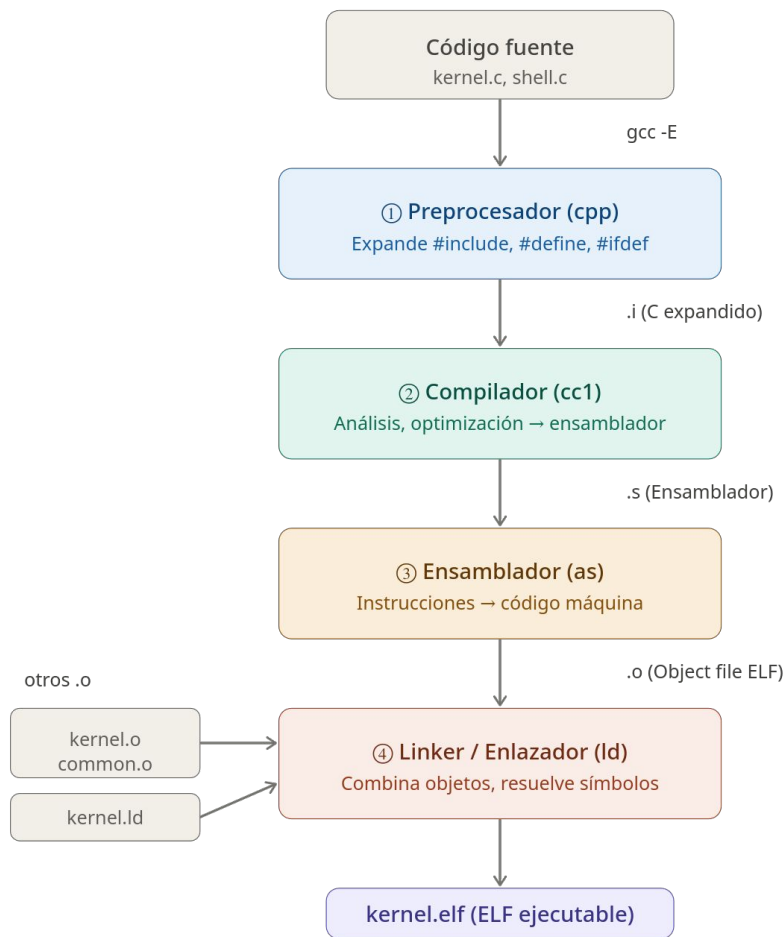
El ensamblador es un paso conceptualmente más simple. Toma el `.s` (texto legible por humanos como `addi a0, a1, 123`) y lo convierte en código máquina binario, produciendo un archivo objeto con formato ELF (`.o`).

Lo que hace:

Convierte cada **mnemónico de instrucción** en su **opcode binario** correspondiente. Resuelve las etiquetas locales (saltos dentro del mismo archivo). Para los símbolos que hacen referencia a cosas definidas en otros archivos (otras funciones, variables globales), deja una entrada de **reubicación (relocation entry)** que dice "acá hay una referencia a `kernel_main`, resuélvela después". También genera la tabla de símbolos del módulo: qué símbolos exporta este `.o` y cuáles necesita del exterior.

El **archivo `.o` resultante tiene estructura ELF**, pero no es ejecutable todavía: las direcciones no están resueltas y los segmentos no están organizados en el layout final de memoria.

El Programa: Compilación



Fase 4 — El Enlazador (ld)

El enlazador toma todos los `.o` y los combina en un único ejecutable ELF final.

Sus tareas principales son:

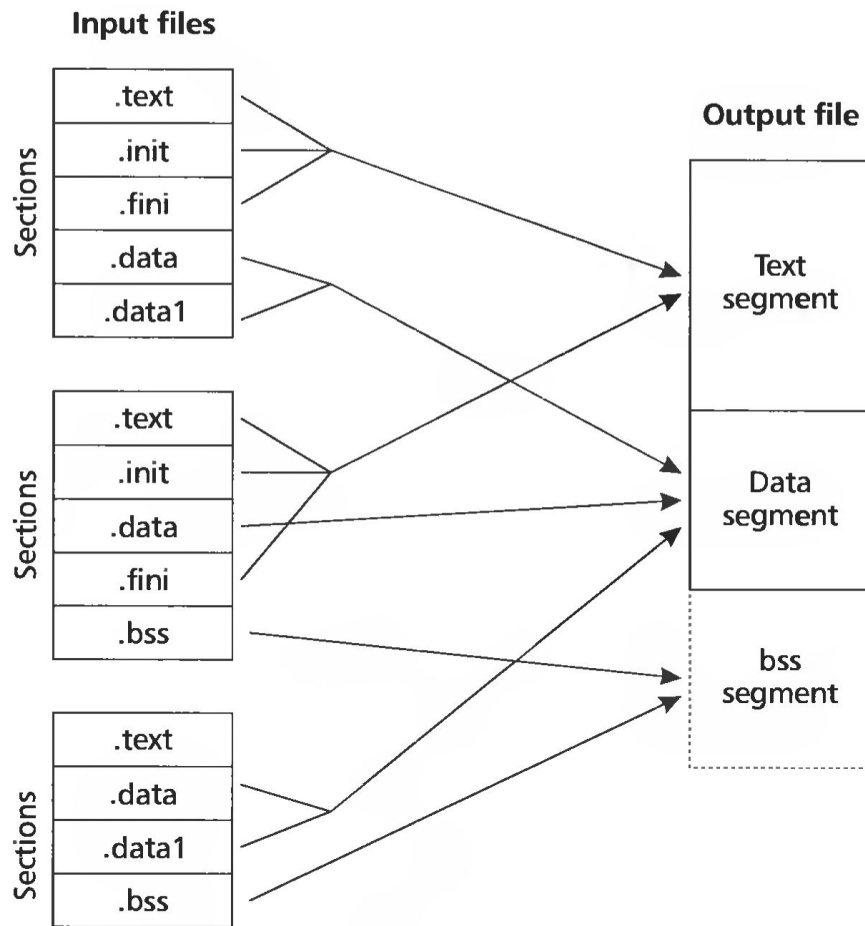
Resolución de símbolos: Cuando `kernel.o` referencia a `printf` que está definida en `common.o`, el linker encuentra esa definición y "conecta" la referencia. Si un símbolo no se encuentra en ningún `.o` ni en las bibliotecas especificadas, el linker arroja un error de "undefined reference".

Reubicación (relocation): Asigna direcciones de memoria definitivas a cada sección y actualiza todas las referencias. ¿El ensamblador dejó entradas pendientes? El linker las resuelve ahora.

Organización según el linker script: En desarrollo de sistemas operativos esto es fundamental. El script de enlazador (`.ld`) le dice al linker exactamente cómo organizar el binario en memoria.

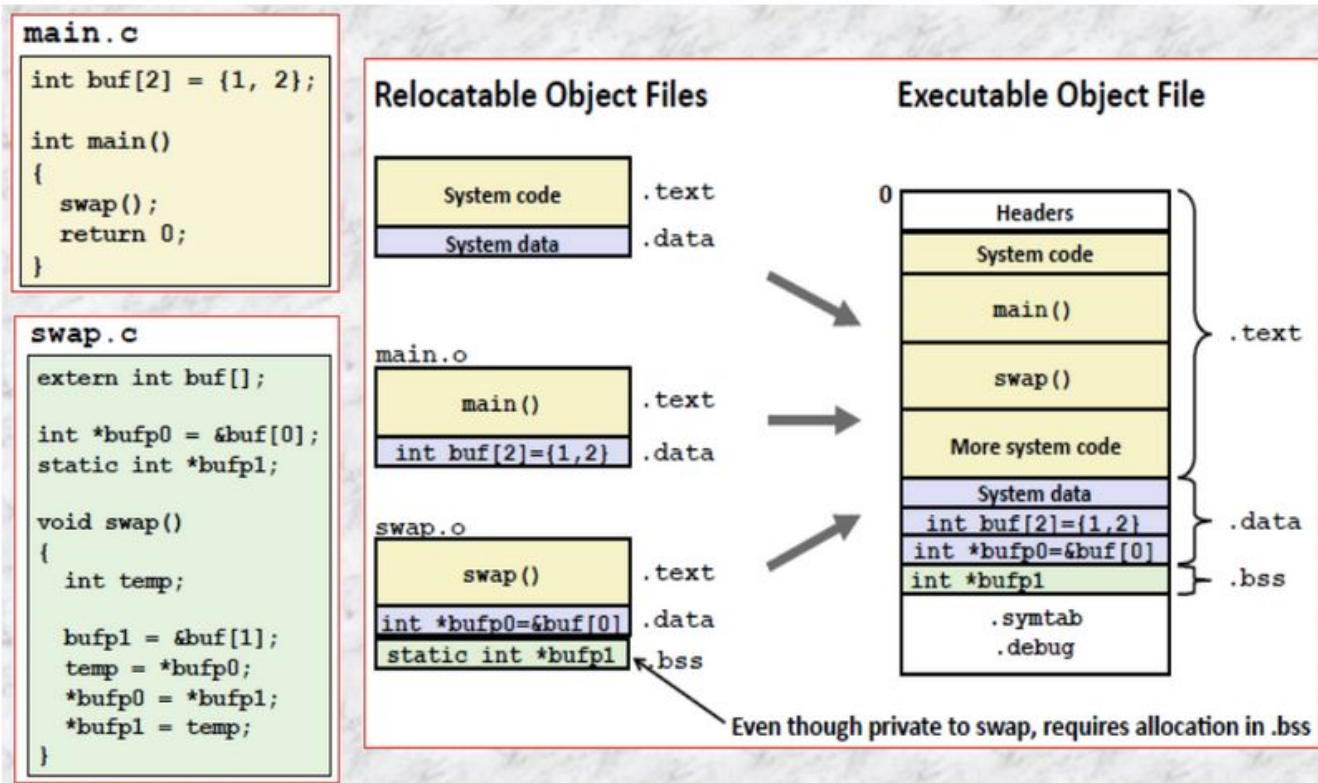
Generación del ELF final: El output es un ELF completamente especificado, con todas las direcciones resueltas y la tabla de segmentos (program headers) lista para ser interpretada por quien cargue el binario (QEMU, el kernel mismo para cargar un proceso de usuario, etc.).

El Programa: Compilación



El proceso de link edicion

El Programa: Compilación



El Programa: Formato Ejecutable

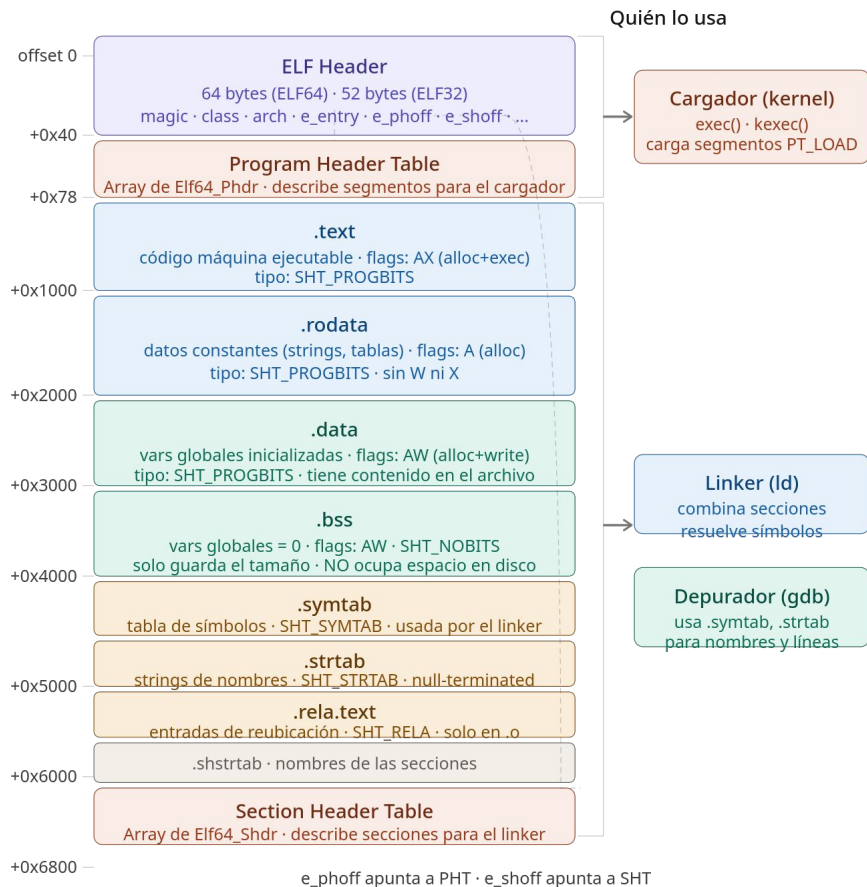
Un *programa* es un archivo que posee toda la información de cómo construir un proceso en memoria [KER](cap. 6).

- **Instrucciones de Lenguaje de Máquina:** Almacena el código del algoritmo del programa.
- **Dirección del Punto de Entrada del Programa:** Identifica la dirección de la instrucción con la cual la ejecución del programa debe iniciar.
- **Datos:** El programa contiene valores de los datos con los cuales se deben inicializar variables, valores de constantes y de literales utilizadas en el programa.

El Programa: Formato Ejecutable

- **Símbolos y Tablas de Realocación:** Describe la ubicación y los nombres de las funciones y variables de todo el programa, así como otra información que es utilizada por ejemplo para debugg.
- **Bibliotecas Compartidas o Dinamicas:** describe los nombres de las bibliotecas compartidas que son utilizadas por el programa en tiempo de ejecución así como también la ruta del linker dinámico que debe ser usado para cargar dicha biblioteca.
- **Otra información:** El programa contiene además otra información necesaria para terminar de construir el proceso en memoria.

El Programa: Formato Ejecutable



Un archivo ELF es una secuencia de bytes con una organización muy precisa. Lo más importante para entenderlo es que el mismo archivo tiene dos lecturas paralelas y complementarias dependiendo de quién lo lee.

La idea central es esta: un archivo ELF es simplemente bytes en disco. Ni más ni menos. Como cualquier archivo. Lo que cambia es con qué "lente" lo mirás.

Los offsets a la Program Header Table y la Section Header Table: Le dicen al lector del ELF dónde encontrar cada tabla dentro del archivo.

El Programa: Formato Ejecutable



Los dos lectores miran los mismos bytes del archivo, pero usan índices diferentes para encontrar lo que les interesa.

Lente del Linker (ld)

Lee la Section Header Table

.text ← código máquina

.rodata ← strings constantes

.data ← variables inicializadas

.bss ← variables en cero

.symtab ← tabla de símbolos

.rela ← reubicaciones pendientes

Pregunta: ¿qué hay acá y cómo lo combino con otros .o?

Lente del Kernel (exec)

Lee la Program Header Table

Segmento 1: LOAD
offset=0x1000 vaddr=0x0 flags=r-x

Segmento 2: LOAD
offset=0x2000 vaddr=0x1000 flags=rw-

Segmento 3: GNU_STACK flags=rw-

(.symtab, .strtab, .rela → ignorados)
(no tienen SHF_ALLOC)

Pregunta: ¿qué copio a RAM
y con qué permisos?

Un archivo ejecutable en Linux: ELF

Ver el ELF Header completo

```
readelf -h kernel.elf
```

Ver todos los segmentos (Program Header Table)

```
readelf -l kernel.elf
```

Ver todas las secciones (Section Header Table)

```
readelf -S kernel.elf
```

Ver la tabla de símbolos

```
readelf -s kernel.elf
```

```
nm kernel.elf
```

Ver las entradas de reubicación de un .o

```
readelf -r kernel.o
```

Disassembly de .text con símbolos

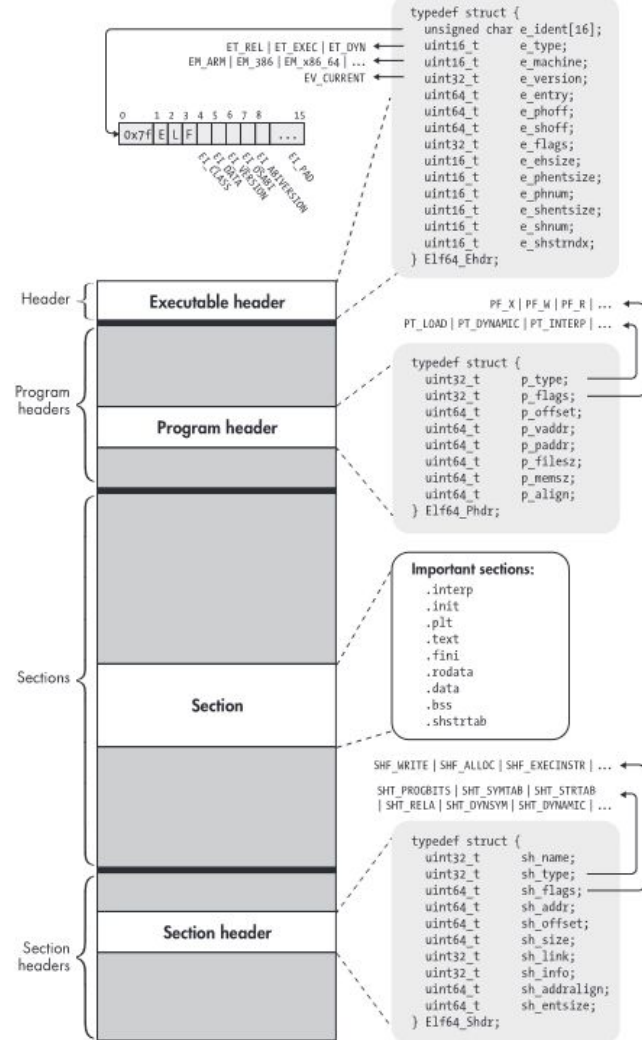
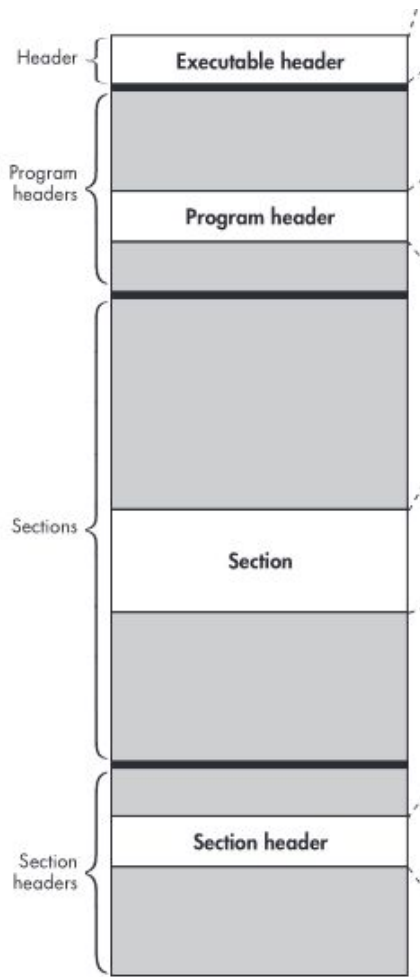
```
objdump -d kernel.elf
```

Ver los primeros bytes en hex (magic number)

```
xxd kernel.elf | head -4
```



Un archivo ejecutable en Linux:



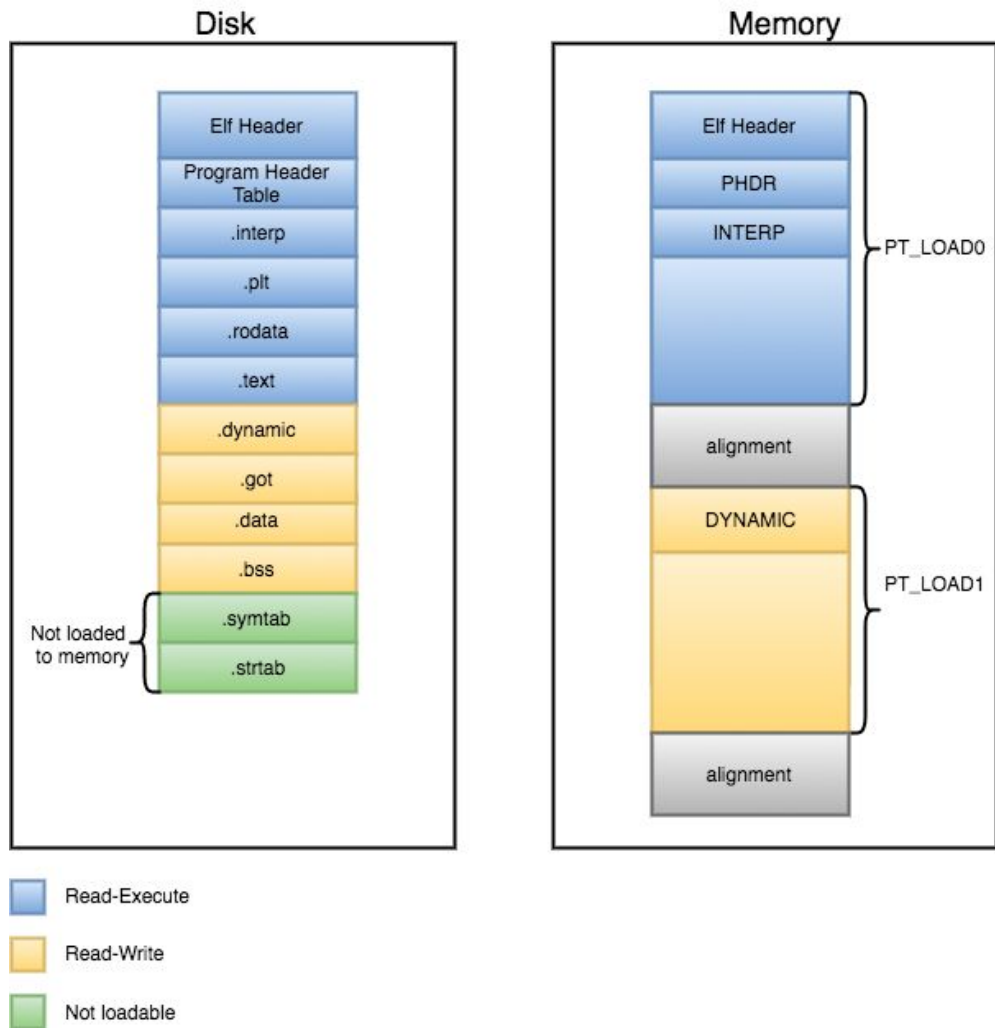
Un Programa en Linux: ELF

```
struct Elf64_Ehdr {  
    unsigned char e_ident[EI_NIDENT];  
    Elf64_Half e_type;  
    Elf64_Half e_machine;  
    Elf64_Word e_version;  
    Elf64_Addr e_entry;  
    Elf64_Off e_phoff;  
    Elf64_Off e_shoff;  
    Elf64_Word e_flags;  
    Elf64_Half e_ehsize;  
    Elf64_Half e_phentsize;  
    Elf64_Half e_phnum;  
    Elf64_Half e_shentsize;  
    Elf64_Half e_shnum;  
    Elf64_Half e_shstrndx;  
};
```

```
struct Elf64_Phdr {  
    Elf64_Word p_type;  
    Elf64_Word p_flags;  
    Elf64_Off p_offset;  
    Elf64_Addr p_vaddr;  
    Elf64_Addr p_paddr;  
    Elf64_Xword p_filesz;  
    Elf64_Xword p_memsz;  
    Elf64_Xword p_align;  
};
```

El archivo ELF además de tener el código, es como un “plano” del proceso que el **Loader** (parte del sistema operativo) va a usar para “construir” el proceso durante **execve()**

Un Programa en Linux: ELF



Un Programa en Linux: ELF

- **readelf** is a Unix binary utility that displays information about one or more ELF files. A free software implementation is provided by GNU Binutils.
- **elfutils** provides alternative tools to GNU Binutils purely for Linux.[11]
- **objdump** provides a wide range of information about ELF files and other object formats. objdump uses the Binary File Descriptor library as a back-end to structure the ELF data.
- The Unix **file** utility can display some information about ELF files, including the instruction set architecture for which the code in a relocatable, executable, or shared object file is intended, or on which an ELF core dump was produced.

El Comando readelf

Ver el SHT (Section Header Table)	<code>readelf -S <archivo></code>
Ver el PHT (Program Header table)	<code>readelf -l <archivo></code>
Ver el ELF Header	<code>readelf -h <archivo></code>
Ver la Relocation Table	<code>readelf -r <archivo></code>
Ver el dump hexa	<code>hexdump -C archivo head -n 10</code>

```

>hexdump -C ./compile_me.elf | head -n 10
00000000  7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00 |.ELF.....|
00000010  02 00 3e 00 01 00 00 00 30 04 40 00 00 00 00 00 |..>....0.@...|
00000020  40 00 00 00 00 00 00 00 00 1a 00 00 00 00 00 00 |@.....|
00000030  00 00 00 00 40 00 38 00 09 00 40 00 1f 00 1c 00 |...@.8...@...|
00000040  06 00 00 00 05 00 00 00 40 00 00 00 00 00 00 00 |.....@.....|
00000050  40 00 40 00 00 00 00 00 40 00 40 00 00 00 00 00 |@.@.....@.@...|
00000060  f8 01 00 00 00 00 00 00 f8 01 00 00 00 00 00 00 |.....|
00000070  08 00 00 00 00 00 00 00 03 00 00 00 04 00 00 00 |.....|
00000080  38 02 00 00 00 00 00 00 38 02 40 00 00 00 00 00 |8.....8.@...|
00000090  38 02 40 00 00 00 00 00 1c 00 00 00 00 00 00 00 |8.@.....|

```

kh3m@kh3m-machine:~/Research/ELF/tests/baseline/compile_options\$

```
>readelf -l ./compile_me.elf | head -n 20
```

Elf file type is EXEC (Executable file)

Entry point 0x400430

There are 9 program headers, starting at offset 64

Program Headers:

Type	Offset FileSiz	VirtAddr MemSiz	PhysAddr Flags	Align
PHDR	0x0000000000000040	0x00000000000400040	0x000000000000400040	
	0x00000000000001f8	0x00000000000001f8	R E	8
INTERP	0x0000000000000238	0x00000000000400238	0x0000000000000400238	
	0x000000000000001c	0x000000000000001c	R	1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]				
LOAD	0x0000000000000000	0x00000000000400000	0x00000000000400000	
	0x000000000000077c	0x000000000000077c	R E	200000
LOAD	0x0000000000000e10	0x00000000000600e10	0x00000000000600e10	
	0x0000000000000228	0x0000000000000230	RW	200000

Risk-V: OpenSBI

Para entender OpenSBI, primero hay que tener claro que RISC-V define tres niveles de privilegio:

Modo	Nombre	¿Quién corre ahí?
M-mode	Machine Mode	OpenSBI (firmware)
S-mode	Supervisor Mode	El kernel del SO
U-mode	User Mode	Las aplicaciones

Anillos de protección

OpenSBI

El flujo de arranque con OpenSBI

Encendido



[M-mode] OpenSBI arranca

- Inicializa la UART (consola serie)
- Detecta el hardware (CPU cores, memoria)
- Configura el timer (CLINT)
- Configura el trap handler en M-mode



[M-mode → S-mode] OpenSBI hace mret

- Salta a la dirección del kernel (ej: 0x80200000)



[S-mode] Tu kernel toma el control

OpenSBI (Open Supervisor Binary Interface) es una implementación de referencia de la especificación SBI (Supervisor Binary Interface), que es básicamente la API que el firmware le ofrece al kernel.

Proceso

Conceptos clave:

- Definición y componentes
- Estructura en memoria del proceso
- System call: `brk()`

El Proceso

“Un proceso es la ejecución de un programa de aplicación con derechos restringidos; el proceso es la abstracción que provee el Kernel del sistema operativo para la ejecución protegida”- [DAH]

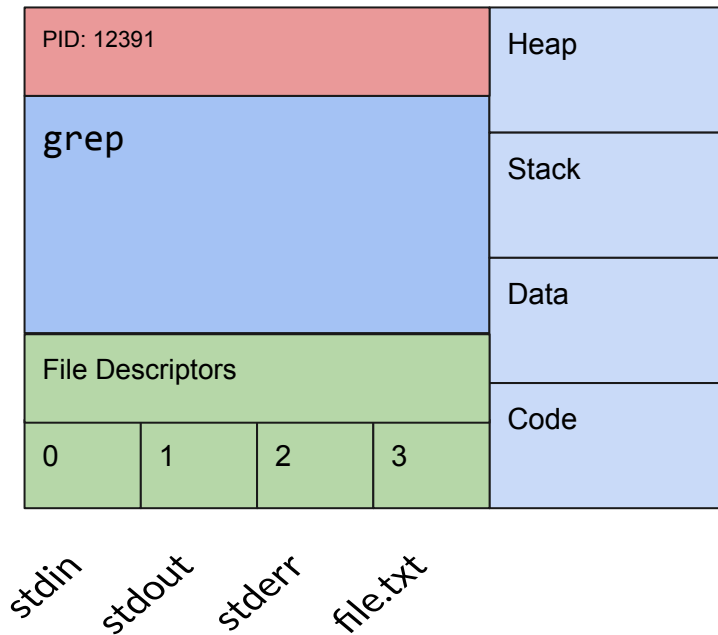
“Es simplemente un programa que se está ejecutando en un instante dado” - [ARP]

“Un Proceso es la instancia de un programa en ejecución” - [VAH]

“Un proceso es un programa en medio de su ejecución” - [LOV]

User Space

El habitante de User Land: **el proceso**



El Proceso

Por supuesto que no está más lejos de la verdad, decir que un proceso es sólo un programa en ejecución. Un proceso incluye:

- Los Archivos abiertos
- Las señales(signals) pendientes
- Datos internos del kernel
- El estado completo del procesador
- Un espacio de direcciones de memoria
- Uno o más hilos de ejecución. Cada thread contiene
 - Un único contador de programa
 - Un Stack
 - Un Conjunto de Registros
 - Una sección de datos globales

“Un **Proceso** es una entidad abstracta, definida por el Kernel, en la cual los recursos del sistema son asignados” [KER]

El Proceso: Virtualización

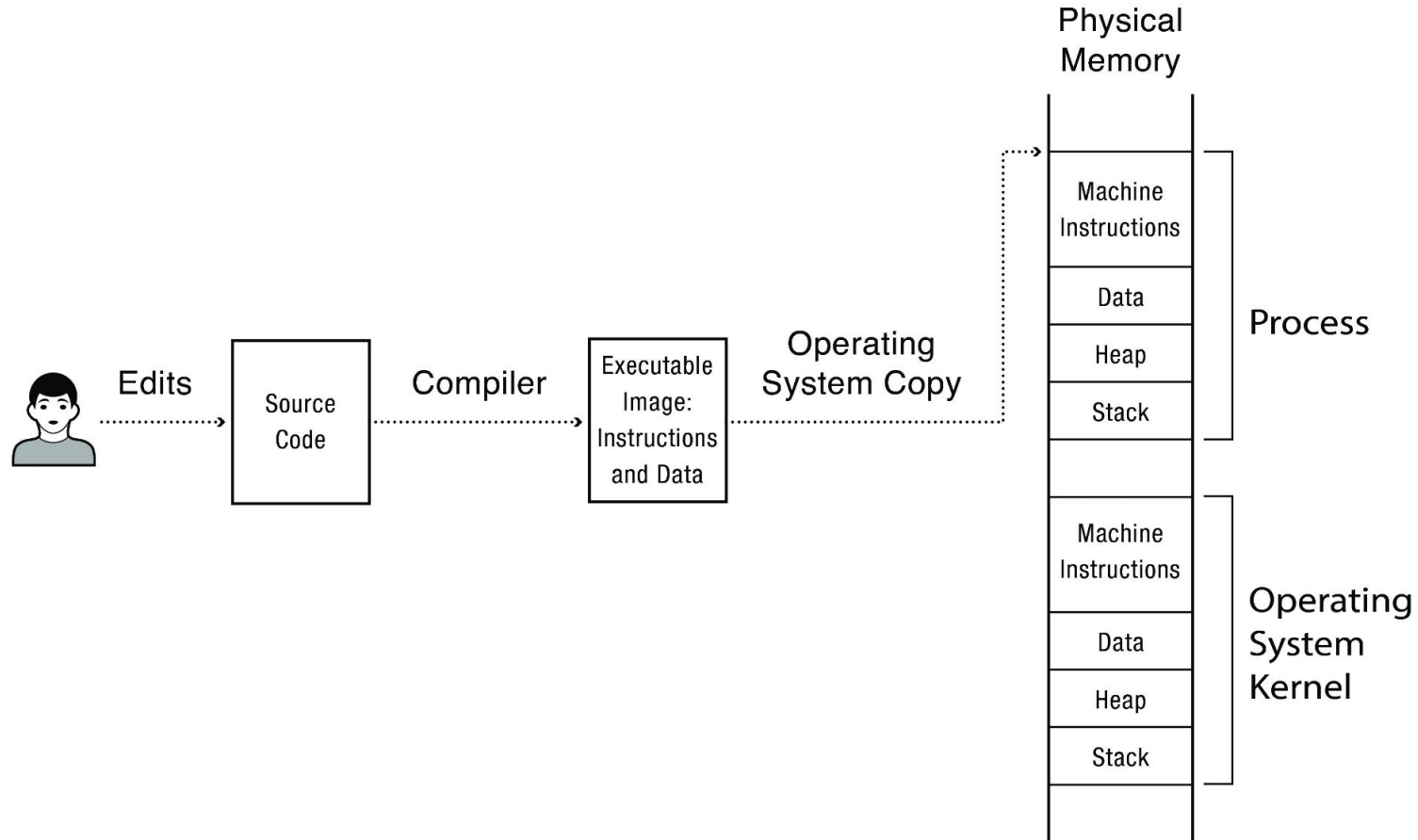
- Virtualización de Memoria
- Virtualización de Procesamiento

De Programa a Proceso

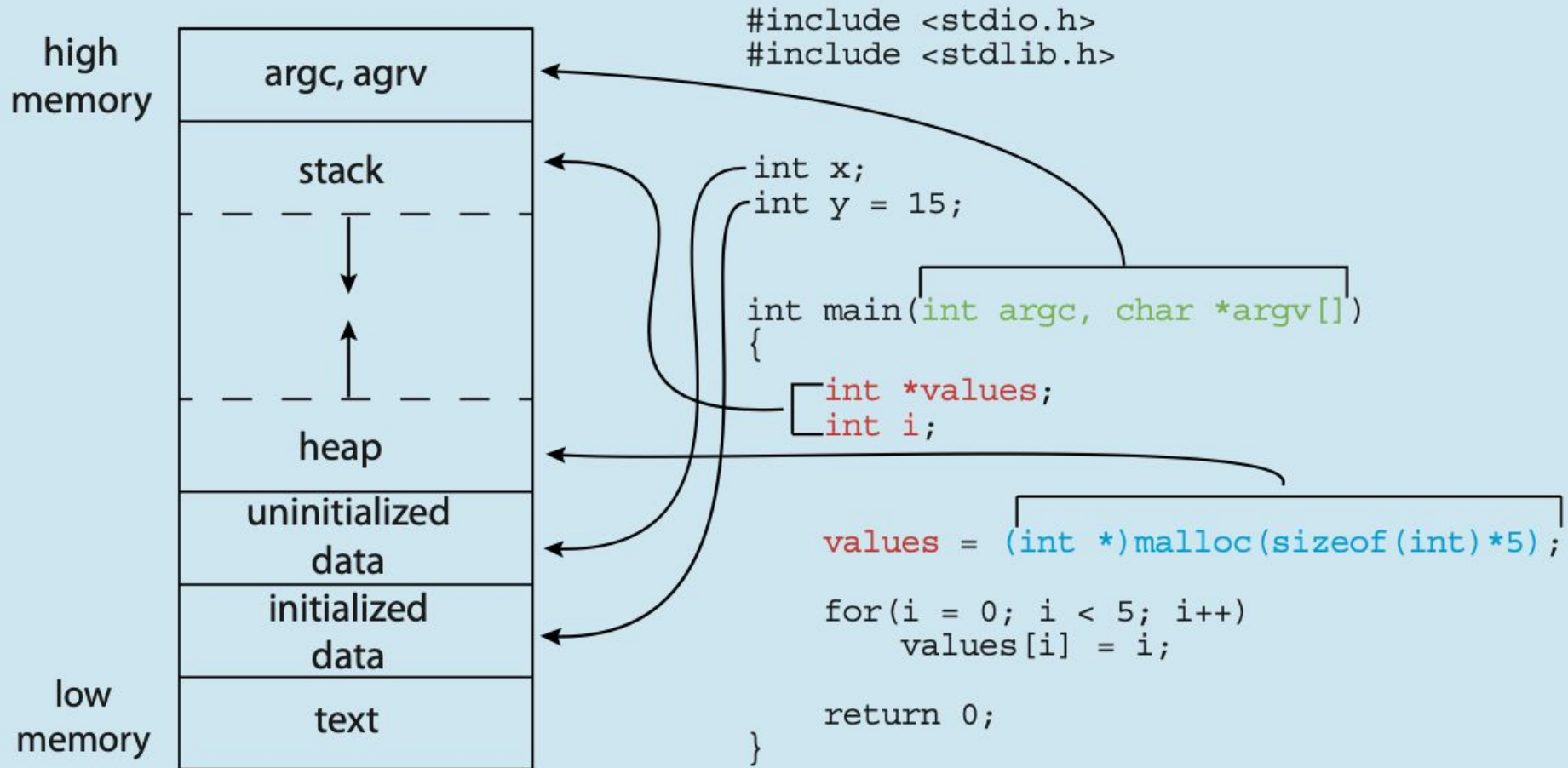
El Sistema Operativo más precisamente el Kernel se encarga de:

1. Cargar instrucciones y Datos de un programa ejecutable en memoria.
2. Crear el Stack y el Heap
3. Transferir el Control al programa
4. Proteger al SO y al Programa

El proceso en memoria

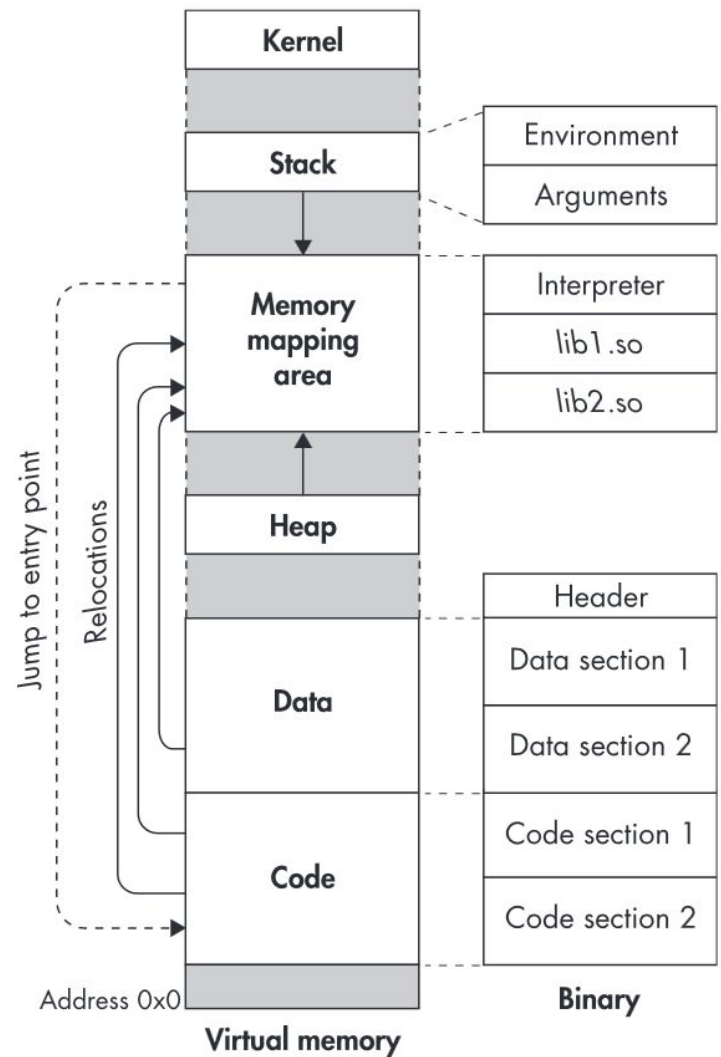


El proceso en memoria



El proceso en memoria

El **Loader** usa el “plano del proceso” que es el archivo ELF y arma el espacio de direcciones



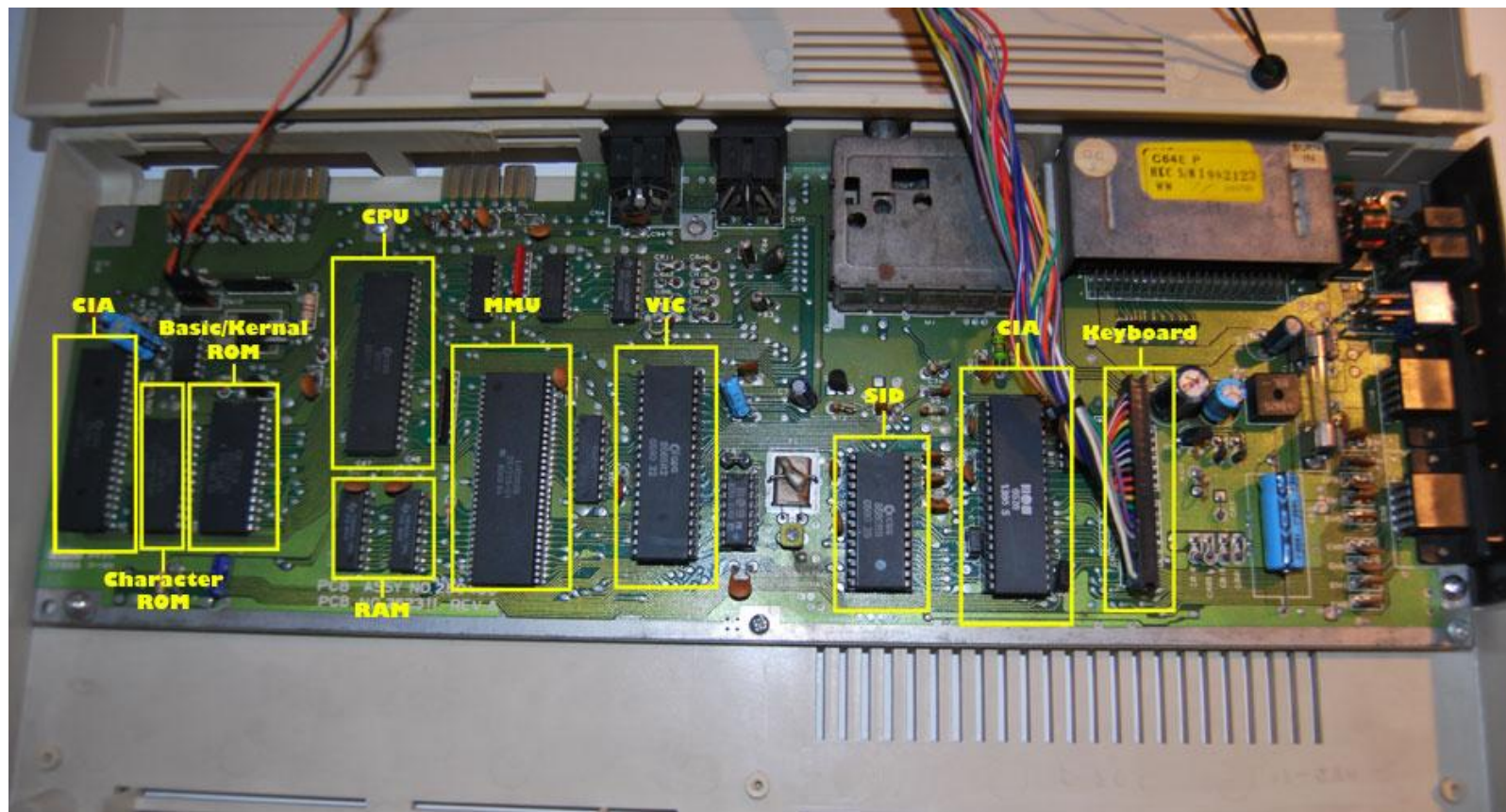
Virtualización de Memoria

- Uno de estos mecanismos es denominado **Memoria Virtual**, la memoria virtual es una abstracción por la cual la memoria física puede ser compartida por diversos procesos.
- Un componente clave de la memoria virtual son las **direcciones virtuales**, con las direcciones virtuales, para cada proceso su memoria inicia en el mismo lugar, la dirección 0.
- Cada proceso piensa que tiene toda la memoria de la computadora para sí mismo, si bien obviamente esto en la realidad no sucede. El hardware traduce la dirección virtual a una dirección física de memoria.

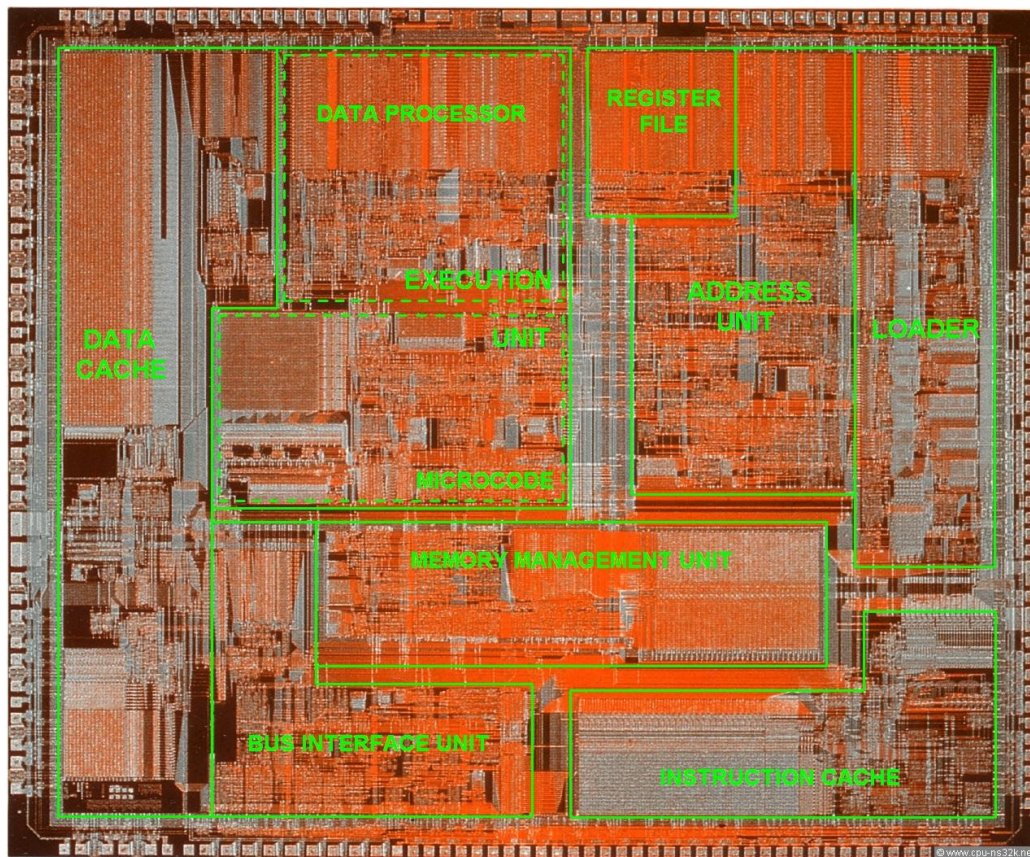
Traducción de Direcciones

Se traduce una Dirección Virtual (emitida por la CPU) en una Dirección Física (la memoria). Este mapeo se realiza por hardware, más específicamente por Memory Management Unit (MMU).



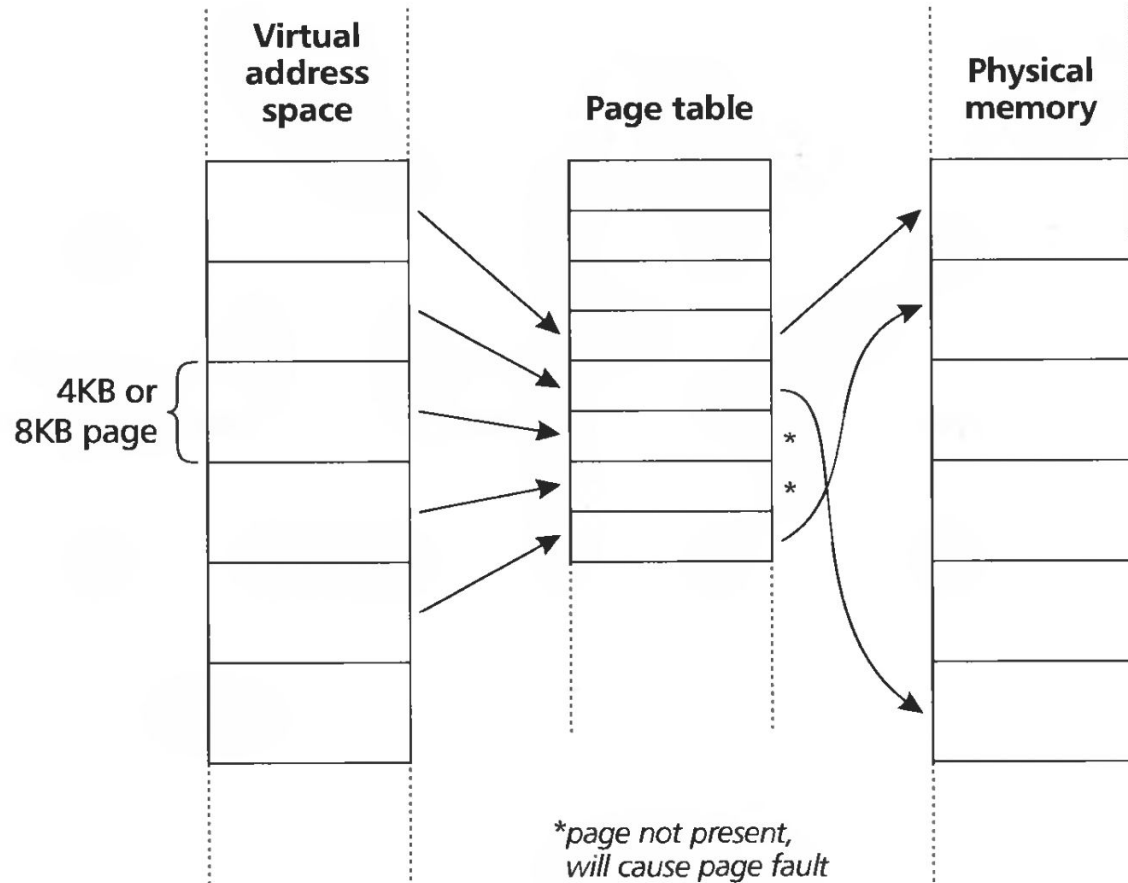


Commodore 64



La MMU Dentro del Procesador

Virtualización de Memoria



El proceso en memoria

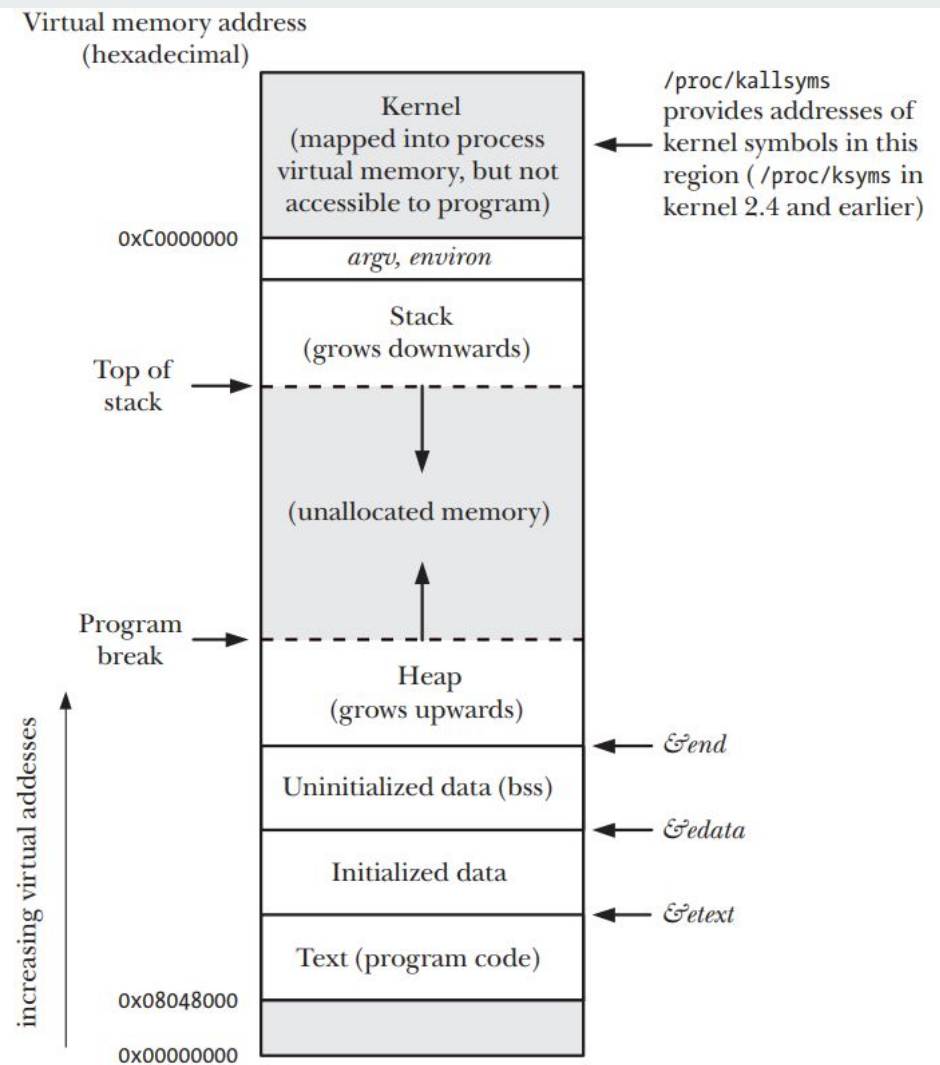
La virtualización de memoria le hace creer al proceso que este tiene toda la memoria disponible para ser reservada y usada como si este estuviera siendo ejecutado sólo en la computadora (ilusión). Todos los procesos en Linux, está dividido en 4 segmentos:

Text: Instrucciones del Programa.

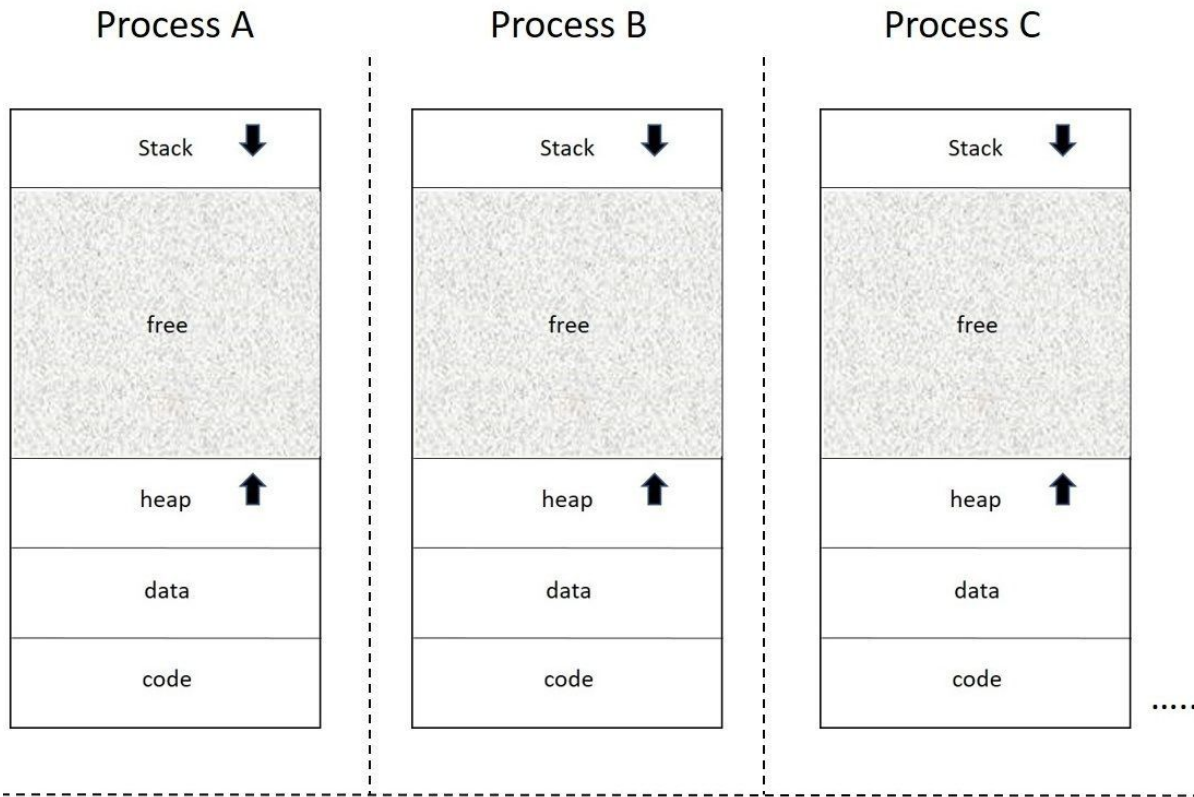
Data: Variables Globales (extern o static en C)

Heap: Memoria Dinámica Alocable

Stack: Variable Locales y trace de llamadas



El proceso en memoria



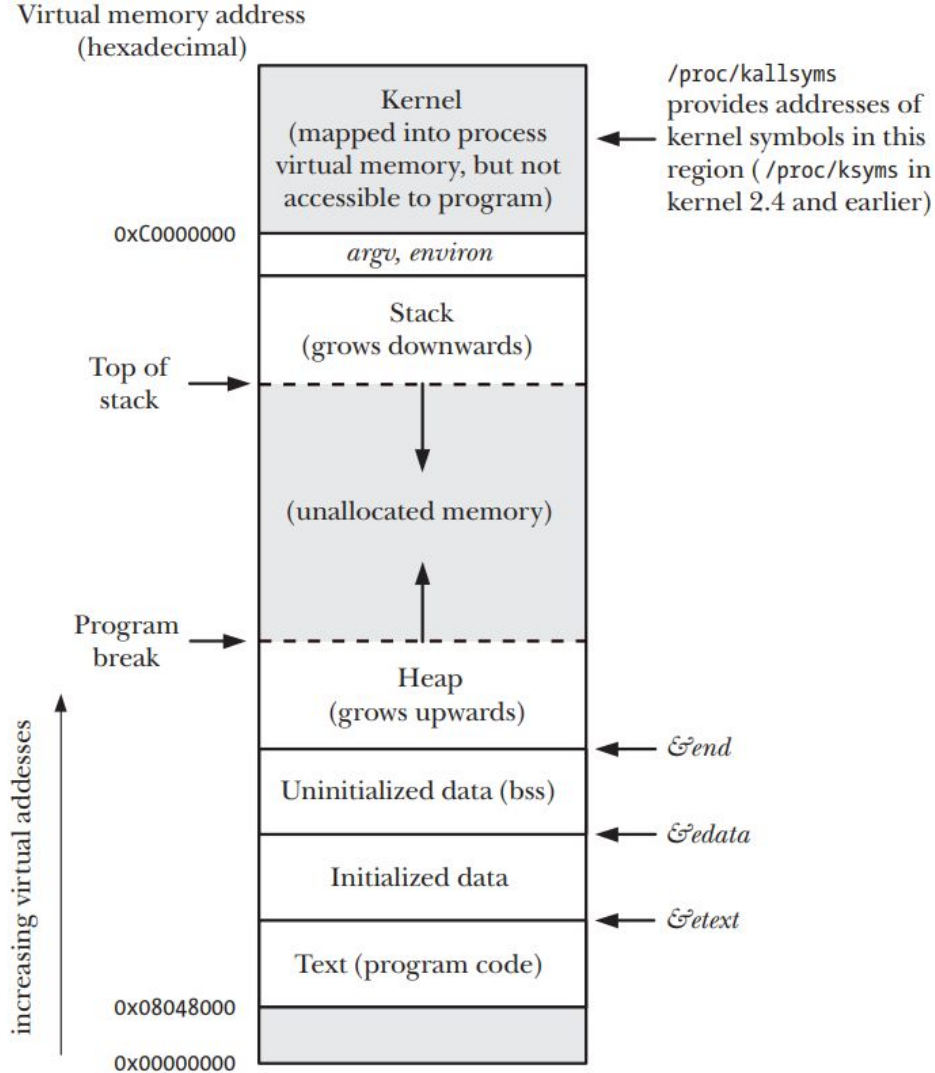
Todos los procesos tienen la misma estructura del Address Space

System call brk()

Inicialmente el break del programa está ubicado justo en el final de los datos no inicializados.

Después que `brk()` se ejecuta, el break es incrementado, el proceso puede acceder a cualquier memoria en la nueva área reservada, pero no accede directamente a la memoria física.

Esto se realiza automáticamente por el kernel en el primer intento del proceso en acceder al área reservada.



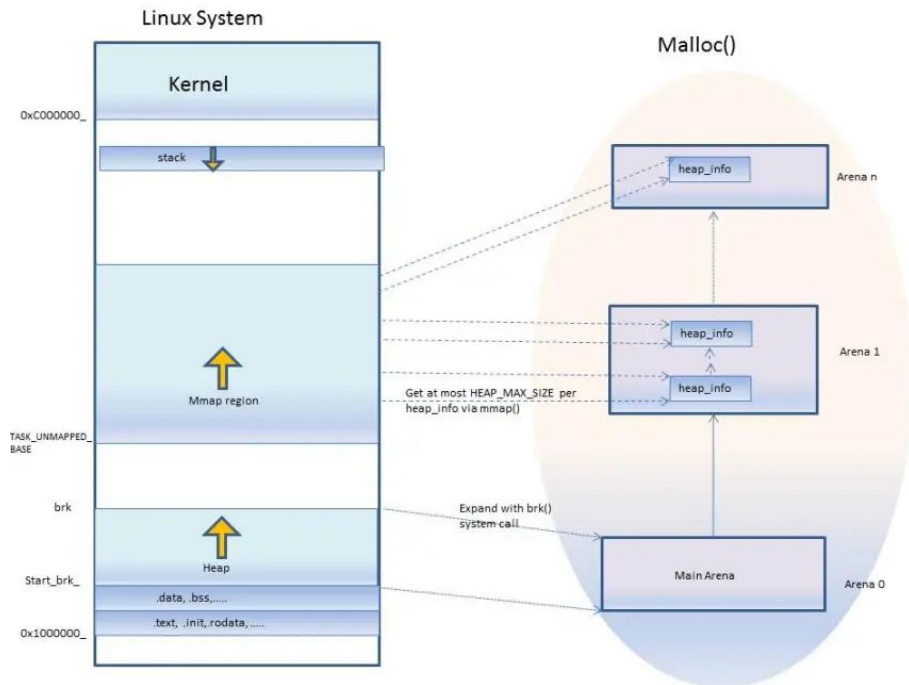
brk() vs malloc()

brk()

- Es una system call. Opera con bloques grandes (generalmente páginas completas)

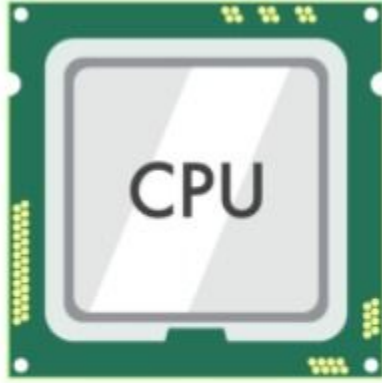
malloc()

- Se implementa en espacio de usuario
- Utiliza brk().
- Maneja estructuras de datos propias (en user space) para optimizar la memoria provista por brk

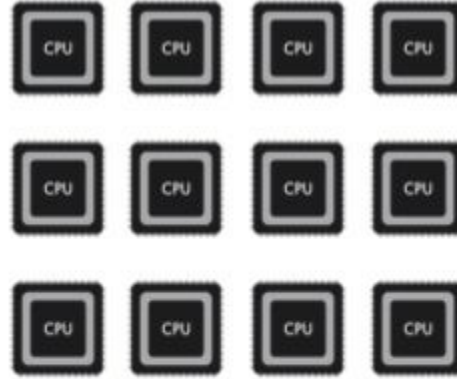


Virtualización de Procesador

La virtualización de procesamiento es la forma de virtualización más primitiva, **consiste en dar la ilusión de la existencia de un único procesador para cualquier programa que requiera de su uso.**

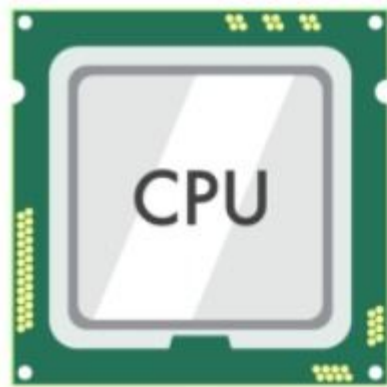


CPU Real



CPU Virtualizada
1 x Proceso

El SO crea esta ilusión mediante la virtualización de la CPU a través del kernel.



CPU Real



CPU Virtualizada



Proceso

-

Abstracción

+

Estados del proceso

Conceptos clave:

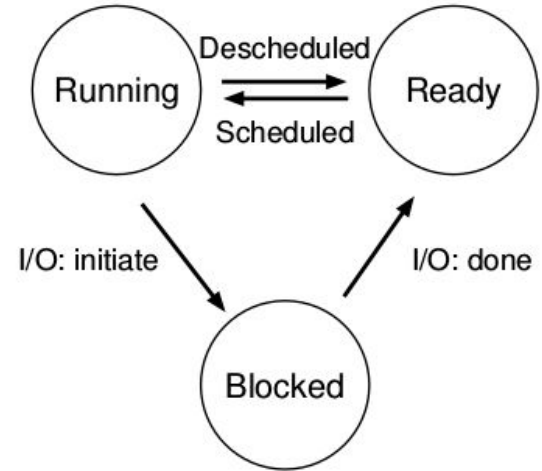
- Diagrama de estados
- Llamadas bloqueantes

Estados de un Proceso

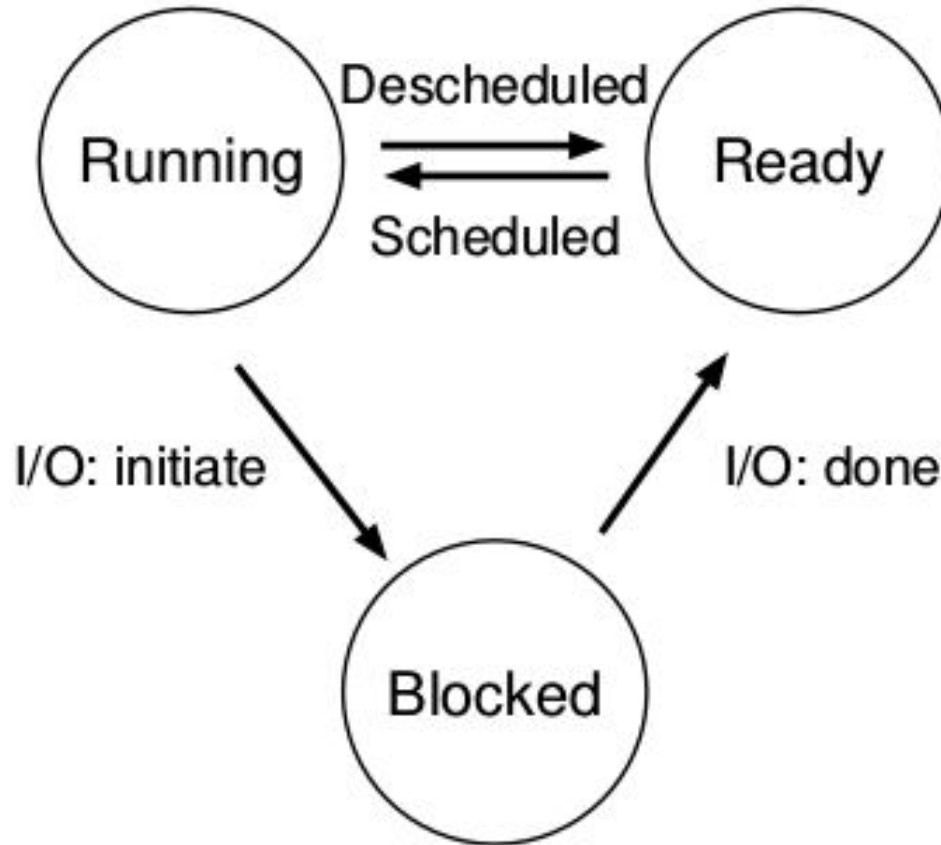
Corriendo (Running): el proceso se encuentra corriendo en un procesador. Está ejecutando instrucciones.

Listo (Ready): en este estado el proceso está listo para correr pero por algún motivo el SO ha decidido no ejecutarlo por el momento.

Bloqueado (Blocked): en este estado el proceso ha ejecutado algún tipo de operación que hace que éste no esté listo para ejecutarse hasta que algún evento suceda.

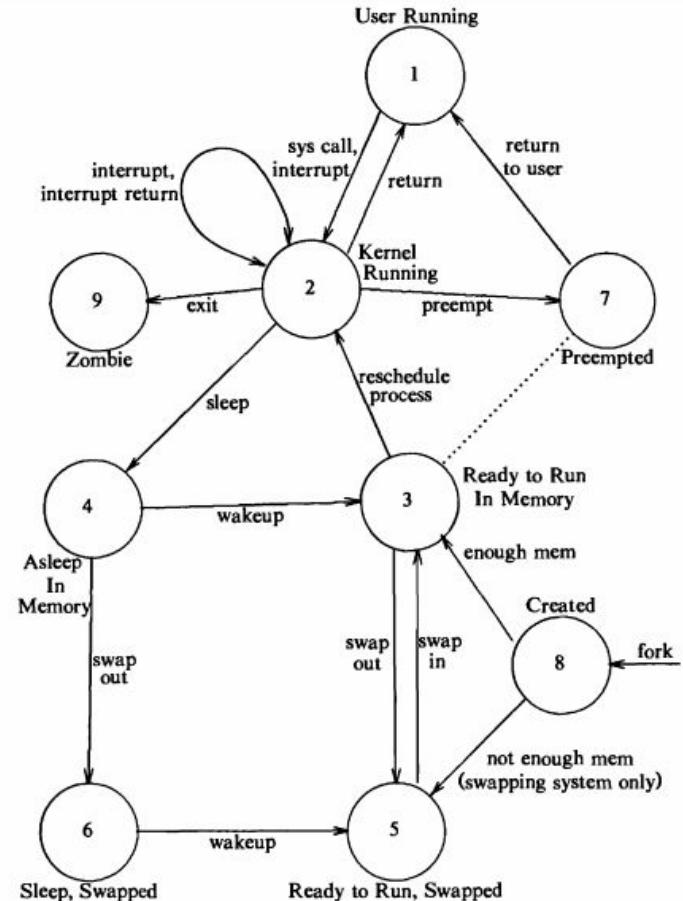


Estados de un Proceso



Ejemplo Histórico: System V

- **Corriendo User Mode(Running User Mode):** El proceso se encuentra corriendo en un procesador. Está ejecutando instrucciones.
- **Corriendo kernel Mode(Running Kernel Mode):** d
- **Listo para Correr en Memoria (Ready to Run on Memory):** En este estado el proceso está listo para correr pero por algún motivo el SO ha decidido no ejecutarlo por el momento.
- **Durmiendo en Memoria (Asleep In Memory) :** El proceso está bloqueado en memoria.
- **Listo para Correr pero Swapeado (Ready to Run but swapped):** El proceso está bloqueado en memoria secundaria.
- **Durmiendo en Memoria Secundaria (Asleep Swapped):** El proceso se encuentra bloqueado en memoria secundaria.
- **Preempt(Preempt):** Es igual a 1 pero un proceso que pasó antes por Kernel mode solo puede pasar a preentive.
- **Creado (Created):** El proceso está recién creado y en un estado de transición.
- **Zombie (Zombie):** El proceso ejecutó la S.C. exit(), ya no existe más, lo único que queda es el exit state.



Estructuras del Kernel

Conceptos clave:

- El contexto del proceso
- [User|Register|System]-Level context
- Kernel Stack

Volvemos al enlazador / linker

¿Qué es un archivo .ld?

Un archivo .ld es un linker script (script de enlazador). Es un archivo de texto que le indica al enlazador (ld, parte de GNU Binutils) cómo debe construir el binario final: qué secciones existirán, en qué orden, y crucialmente, en qué dirección de memoria virtual deben ubicarse.

El enlazador toma los .o (object files) generados por el compilador y, guiado por el .ld, produce el ejecutable final (.elf, por ejemplo).

Volvemos al enlazador / linker

```
ENTRY(boot)          /* 1. Punto de entrada */

SECTIONS {
    . = 0x80200000; /* 2. Location counter - dirección base */

    .text : {
        KEEP(*( .text.boot )); /* 3. Forzar primero el código de arranque */
        *( .text .text.* );
    }
    .rodata : ALIGN(4) { *( .rodata .rodata.* ); }
    .data : ALIGN(4) { *( .data .data.* ); }
    .bss : ALIGN(4) {
        __bss = .; /* 4. Símbolo exportado al código C */
        *( .bss .bss.* .sbss .sbss.* );
        __bss_end = .;
    }
    . = ALIGN(4);
    . += 128 * 1024; /* 5. Reservar 128KB para el stack */
    __stack_top = .; /* 6. Símbolo del tope del stack */
}
```

Ejemplo de
archivo ld

Proceso: El Contexto

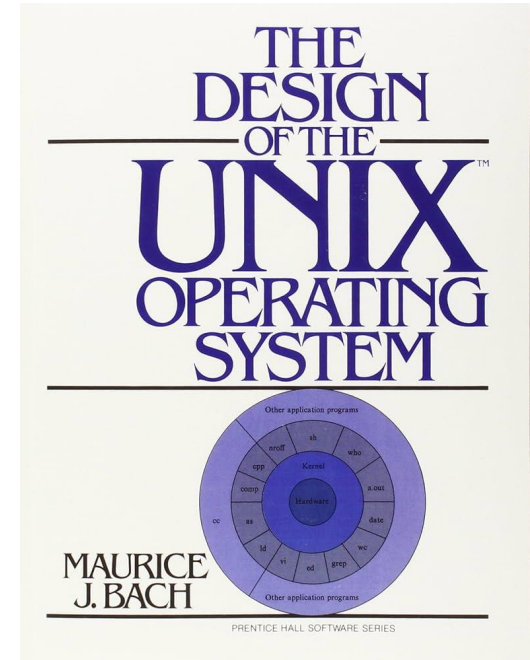
- El contexto de un proceso es la información necesaria para describir completamente el estado de un proceso.
- Cada proceso posee su propio contexto.

Proceso: El Contexto

¿Qué se necesita para describir “el estado de un proceso”?

Según Bach: el contexto de un proceso comprende:

- el contenido del address space (**memoria**)
- el contenido de los registros de hardware (**cpu**)
- las estructuras de datos que pertenecen al kernel relacionadas con el proceso (**kernel**)



Proceso: El Contexto

Formalmente, el contexto de un proceso es la unión de:

- User-level context
- Register context
- System-level context

Proceso: El Contexto

Formalmente, el contexto de un proceso es la unión de:

- User-level context (**memoria**)
- Register context (**cpu**)
- System-level context (**estructuras del kernel**)

Cada proceso ve solamente su contexto, ie. su memoria, su procesador, sus archivos abiertos, etc.

Proceso: El Contexto User-level

- Consiste en las secciones que conforman el espacio de direccionamiento virtual del proceso
 - Text
 - Data
 - Stack
 - Heap

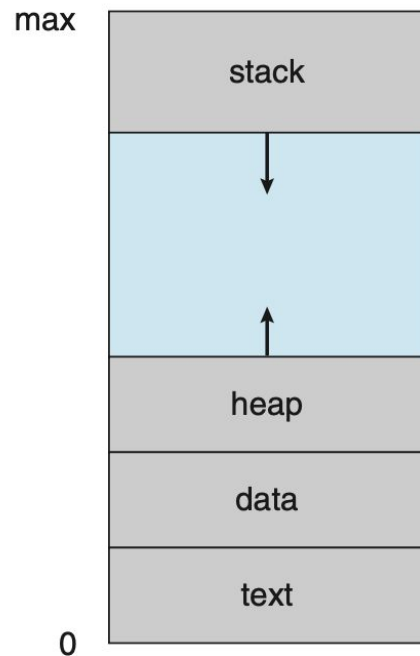


Figure 3.1 Layout of a process in memory.

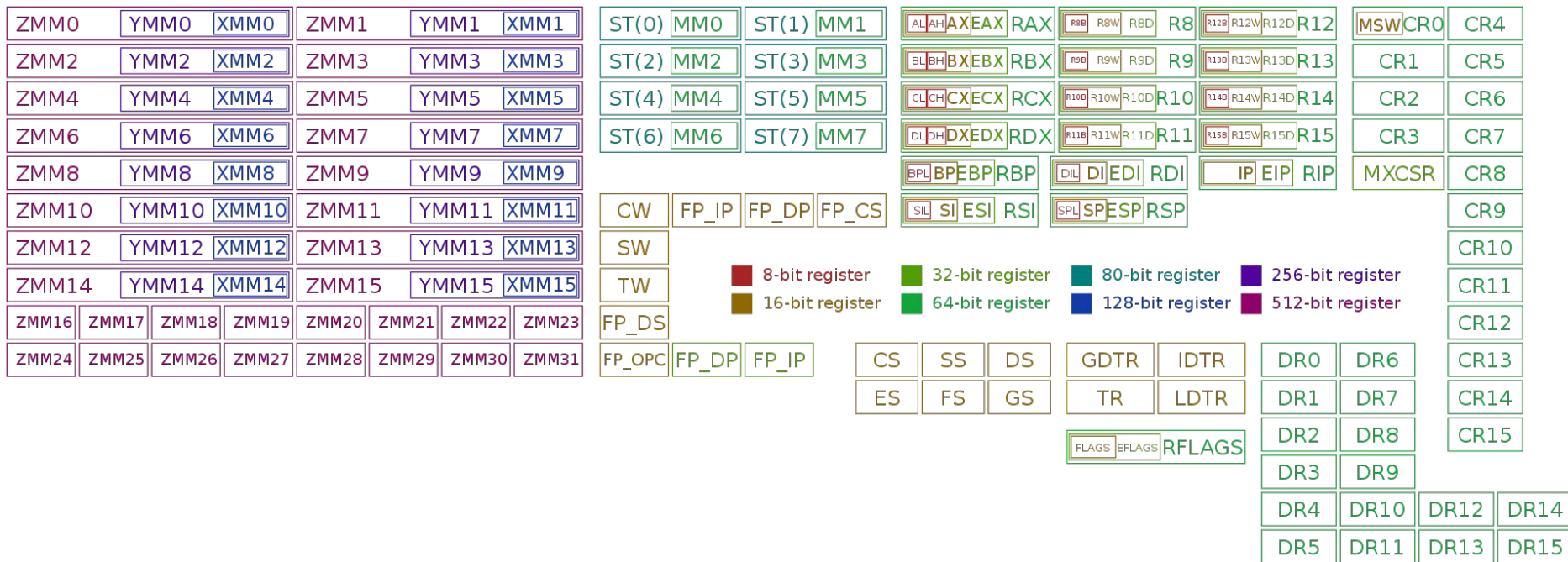
Proceso: El contexto de registros

Representa el estado del CPU, es decir, los registros del CPU

Eg. x86

- Program Counter Register
- Processor Status Register
- Stack Pointer Register
- General Purpose Registers

Proceso: El contexto de registros



System-level context

- **Process Table Entry:** La entrada en la **tabla de procesos**
- **Configuración de memoria:** define el mapeo de la memoria virtual vs memoria física del proceso. Eg. Page Tables
- **Kernel Stack.** contiene los stack frames de las llamadas a funciones hechas dentro del kernel.
- **La u area:** en desuso en sistemas modernos. Pero casi todo se incluye ahora en la Process Table Entry

System-level context: U Area

La u-area era una estructura por proceso usada en sistemas antiguos (UNIX Version 6) para almacenar información específica de cada proceso, como descriptores de archivos abiertos, registros en modo usuario y manejo de señales.

Con el tiempo, los sistemas Unix y similares, como Linux, han evolucionado y reestructurado la gestión de procesos.

En general la Process Table Entry contiene todo lo que antes estaba en la u-area

System-level context: Process Table Entry (o PCB)

Cada proceso está representado en el sistema operativo por un Process Control Block (PCB) .
Un PCB se muestra en la Figura 3.3.

Contiene muchos elementos de información asociados con un proceso específico.

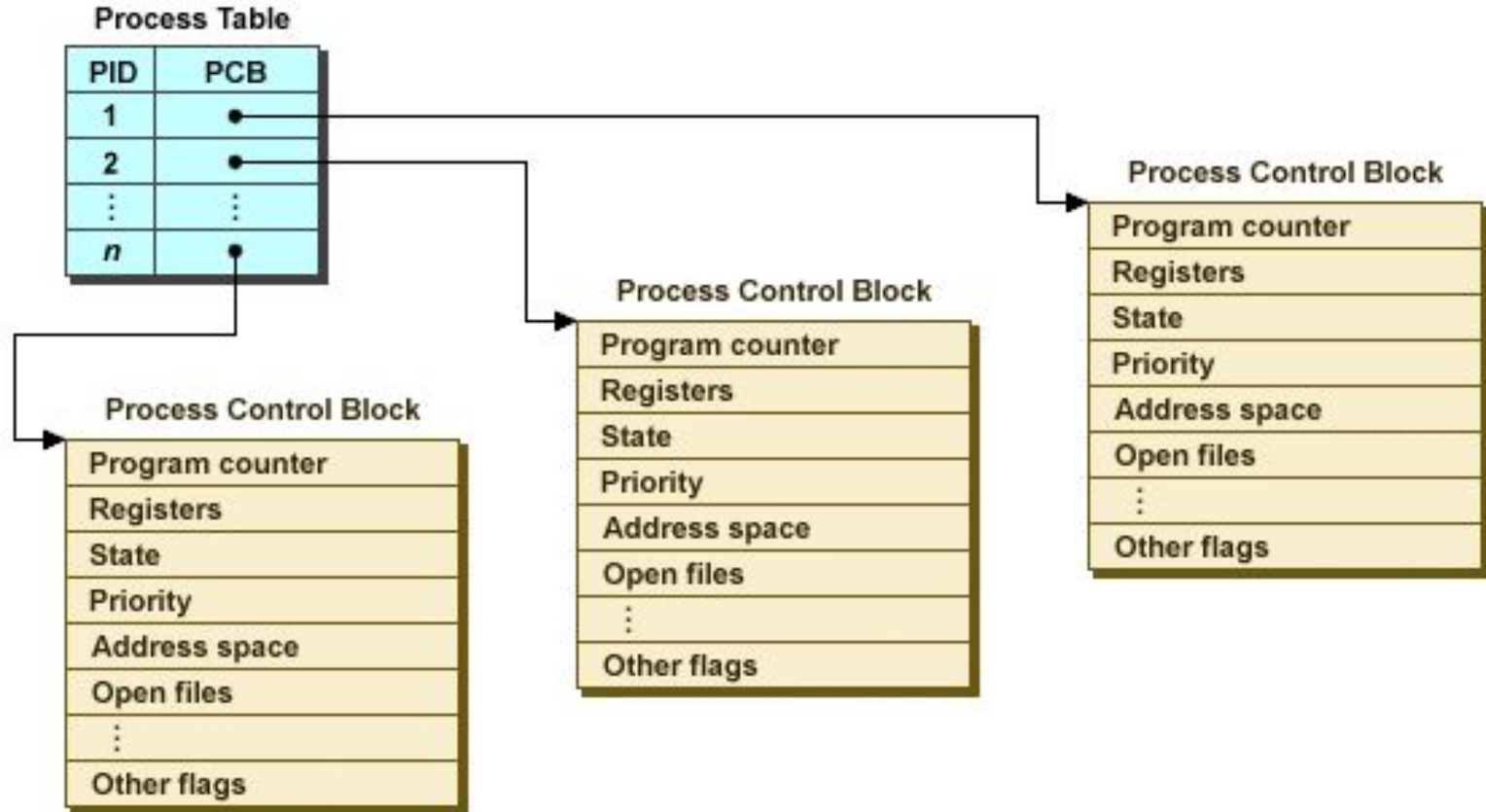
Como se guardan en una tabla, se suelen llamar también Process Table Entry



Figure 3.3 Process control block (PCB).

El Contexto System-level: Process Table

La tabla de PCB es generalmente un array o una lista enlazada



System-level context: Process Table Entry

Depende mucho de la implementación del sistema operativo. Cosas que **podría** contener el PCB:

- identificación: cada proceso tiene un identificador único o process ID (PID) y además perteneces a un determinado grupo de procesos.
- Ubicación del mapa de direcciones del Kernel del u area del proceso.
- Estado actual del proceso
- Un puntero hacia el siguiente proceso en el planificador y al anterior.
- Prioridad
- Información para el manejo de señales.
- Información para la administración de memoria.

Ejemplo PCB: struct proc (xv6)

En xv6 la tabla de procesos es simplemente un array de 64 entradas

kernel/param.h

```
#define NPROC          64    // maximum number of processes
```

kernel/proc.c

```
struct proc proc[NPROC];
```

Ejemplo PCB: struct proc (xv6)

Cada PCB tiene toda la información del proceso

kernel/proc.h

```
struct proc {  
    ...  
    enum procstate state;           // Process state  
    int xstate;                     // Exit status to be returned to parent's wait  
    int pid;                        // Process ID  
    struct proc *parent;            // Parent process  
    ...  
}
```

Ejemplo PCB: struct proc (xv6)

kernel/proc.h (continuación)

```
...

uint64 kstack;                // Virtual address of kernel stack

pagetable_t pagetable;        // User page table

struct file *ofile[NOFILE];   // Open files

struct inode *cwd;             // Current directory

char name[16];                 // Process name (debugging)

...

};
```

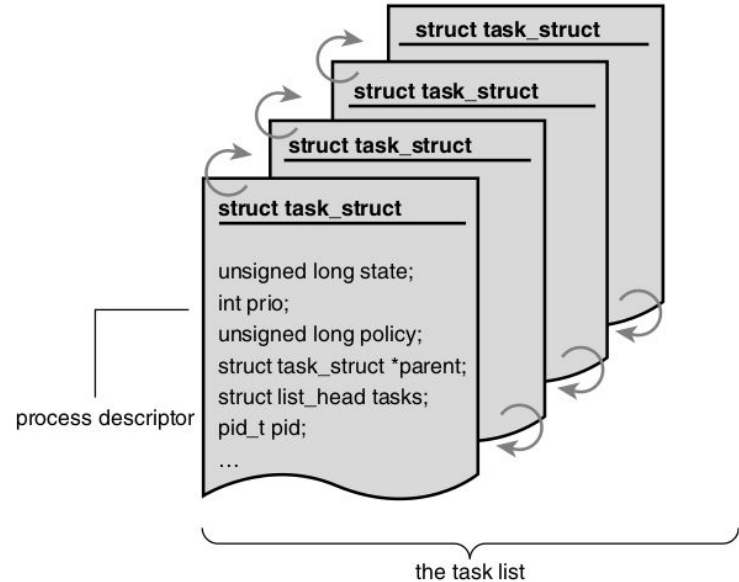
Ejemplo PCB: struct proc (xv6)

Resumiendo. Cada PCB en xv6 tiene:

- Process Id
- Nombre
- Process State
- Exit state
- Parent Process
- Open files
- Current directory
- Page Table
- Kernel Stack
- Estructuras auxiliares para: context switch, locking

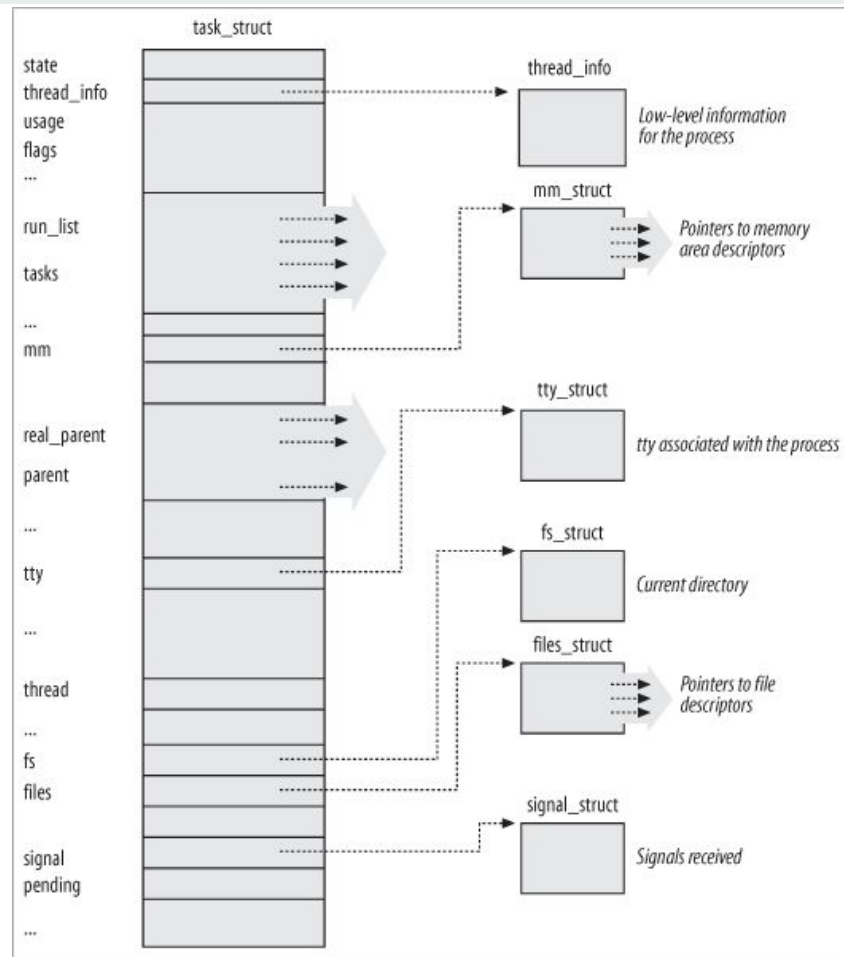
Ejemplo PCB: task_struct (Linux)

El kernel almacena la lista de procesos en una lista circular doblemente enlazada llamada **task list**.



Ejemplo PCB: task_struct (Linux)

La task_struct en sí es una estructura compleja en Linux.



Anticipo: Context Switch

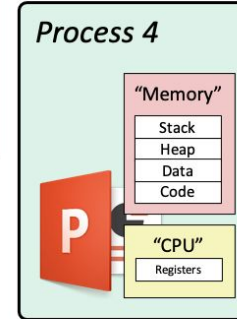
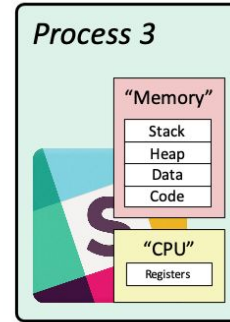
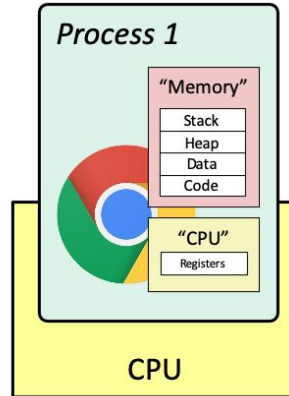
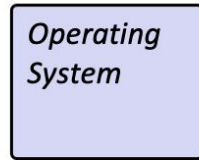
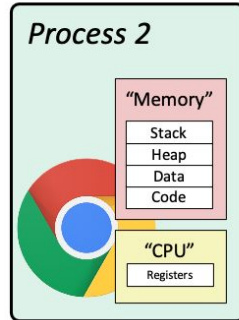
Spoiler alert! El context switch

¿Como se cambia de contexto, de un proceso a otro?

- Se cambia el User-level context (**memoria**)
- Se cambia el Register context (**cpu**)
- Se cambia el System-level context (**estructuras del kernel**)

Hágalo muy rápido y obtendrá la ilusión de concurrencia.
Cientos de procesos ejecutándose en un solo procesador!

Computer



Disk

/Applications/



Chrome.exe

Slack.exe

PowerPoint.exe

Kernel stack

El Kernel usa funciones como cualquier programa, pero no puede usar el stack del user space para implementar sus funciones:

- No es seguro escribir datos del kernel en el user space
- Podria romper el user space (no hay garantías de como el user usa el stack)

El Kernel va a tener su propio stack, separado, en la zona de memoria del Kernel, protegida del usuario.

Kernel stack

¿Por qué un kernel stack por proceso (y no uno solo global)?

Porque cada proceso “dormido” podía estar haciendo cosas completamente diferentes en el espacio Kernel antes de dormirse:

Eg.

- Un proceso se bloquea ejecutando read()
- Un proceso se bloquea al hacer sleep()
- Un proceso se bloquea mientras configuraba mas memoria virtual

Kernel stack

Cuando se hace un context-switch, pasan cosas:

- Se restauran los registros del proceso entrante (EAX, EBX, etc.)
- Se cambia toda la zona de usuario por la del proceso entrante (stack, heap, text, data)
- No se modifica la memoria del Kernel, porque el Kernel es común y compartido a todos los procesos. **Pero:**
- Se apunta el stack pointer al kernel stack del proceso entrante.